

# Python File & Directory Handling

## os and pathlib Modules — A Practical Handout for Python Students

University Academic Handout • Computer Science & Engineering • Python 3 Standard Library

### Learning Objectives

By completing this study handout, students will be able to:

- **1. Understand Filesystem Paths:** Distinguish between absolute and relative paths across operating systems (Windows, macOS, Linux).
- **2. Navigate Directories Programmatically:** Inspect and alter the current working directory safely inside Python applications.
- **3. Manage Files and Folders:** Create nested directory trees, rename, move, and remove files safely without crashes.
- **4. Search and Filter Files:** Utilize glob and rglob wildcard patterns to discover datasets, images, and model checkpoints across complex hierarchies.
- **5. Handle Text Files Natively:** Read and write text files cleanly using modern object-oriented methods.
- **6. Manage Configuration Secrets:** Read environment variables via `os.getenv()` rather than hardcoding sensitive credentials in source code.
- **7. Choose the Right Tool:** Understand when to use the modern, object-oriented `pathlib` and when to use the low-level `os` module.

### 1. Introduction

Every computer relies on an Operating System (OS)—such as Windows, macOS, or Linux—to manage hardware, coordinate memory, and organize persistent storage into files and folders.

As a developer, data scientist, or machine learning practitioner, writing code that only operates on in-memory variables (such as integers, lists, or dictionaries) is rarely sufficient. Real-world applications constantly interact with disk storage:

- **Data Science & Machine Learning:** Loading raw CSV/Parquet datasets, saving cleaned feature matrices, and writing trained neural network weights (.pt, .h5, .pkl).
- **Artificial Intelligence & Computer Vision:** Iterating through nested directories holding tens of thousands of image files, annotations, and bounding box logs.
- **Backend & Web Development:** Handling user file uploads, writing server logs, and loading environment configurations securely.

### Why We Need Both `os` and `pathlib`

The Python standard library provides two primary tools for handling files, directories, and the operating system:

- **os (Operating System Module):** Python's long-standing, low-level module that provides direct access to operating system services. It handles paths strictly as raw strings (e.g., 'data/raw/cats.csv') and exposes process-level OS features such as environment variables and changing the process working directory.
- **pathlib (Object-Oriented Filesystem Paths):** Introduced in Python 3.4 (PEP 428) as the modern standard for path manipulation. Instead of treating paths as plain strings prone to syntax errors, pathlib treats paths as rich objects with built-in properties and methods.

**NOTE: Modern Python Best Practice**

pathlib does not completely replace os; rather, they complement each other. pathlib is the preferred choice for all filesystem paths, directory navigation, and file reading/writing. The os module remains essential for process-level operations such as environment variables (os.getenv) and changing process execution directories (os.chdir).

## Sample Project Hierarchy

Throughout this handout, we will frequently refer to a typical real-world Machine Learning project structure:

```
AI_Project/  
├── data/  
│   ├── raw/  
│   │   ├── train_cats.csv  
│   │   └── train_dogs.csv  
│   └── processed/  
├── models/  
│   └── checkpoints/  
└── notebooks/  
    └── exploration.ipynb
```

Python programs can programmatically verify whether folders exist, create the missing checkpoint directories automatically, loop over thousands of raw files, and save cleaned outputs—all without manual human intervention.

## PART A — os MODULE

The os module is part of Python's standard library. It provides operating system dependent functionality, allowing your script to interface directly with the underlying platform.

### 2. Importing the os Module

Because os is built into Python, no third-party installation (like pip) is needed. You import it directly at the top of your script:

```
import os
```

### 3. Current Working Directory: `os.getcwd()`

What does 'Current Working Directory' (CWD) mean? The CWD is the folder from which your Python script is executed inside your terminal or IDE. When your script references a relative file name like 'data.csv', Python looks for that file inside the current working directory.

To inspect where Python is currently looking on your filesystem, call `os.getcwd()`:

```
import os

cwd = os.getcwd()
print("Current Working Directory:", cwd)
```

#### Expected Output:

```
Current Working Directory: /home/user/AI_Project
# On Windows, this would look like: C:\Users\Student\AI_Project
```

### 4. Changing the Working Directory: `os.chdir()`

While `os.getcwd()` inspects your location, `os.chdir(path)` physically shifts Python's working directory to a new folder for the duration of the running process.

```
import os

print("Starting in:", os.getcwd())

# Move into the 'data' folder
os.chdir("data")
print("Changed directory to:", os.getcwd())

# Move back up one level using '..'
os.chdir("..")
print("Returned back to:", os.getcwd())
```

#### Expected Output:

```
Starting in: /home/user/AI_Project
Changed directory to: /home/user/AI_Project/data
Returned back to: /home/user/AI_Project
```

#### 🔗 PRO TIP: Key Distinction

`os.getcwd()` is a passive 'getter' that inspects your current directory. `os.chdir()` is an active 'setter' that alters the process's working directory. In large production codebases, changing the CWD globally is often discouraged because it can unexpectedly break relative paths in other modules.

## 5. Listing Files and Directories: `os.listdir()`

To see what contents exist inside a directory, use `os.listdir()`. If you provide no argument, it lists the contents of the current directory. If you provide a path argument, it lists that specific directory.

```
import os

# List contents of current directory
all_items = os.listdir()
print("Current folder contents:", all_items)

# List contents of a specific subfolder
data_items = os.listdir("data")
print("data/ folder contents:", data_items)
```

### Expected Output:

```
Current folder contents: ['data', 'models', 'notebooks', 'README.md']
data/ folder contents: ['raw', 'processed']
```

## 6. Checking Whether Something Exists: `os.path.exists()`

Before opening or processing a file, you should always verify if it actually exists. Attempting to open a missing file will raise a `FileNotFoundError`. `os.path.exists()` returns `True` if the file or folder exists, and `False` otherwise.

```
import os

file_to_check = "data/raw/train_cats.csv"

if os.path.exists(file_to_check):
    print(f"File found: {file_to_check}. Ready for training.")
else:
    print(f"Error: {file_to_check} does not exist! Please verify data pipeline.")
```

### Expected Output:

```
File found: data/raw/train_cats.csv. Ready for training.
```

## 7. Checking File vs Directory: `os.path.isfile()` and `os.path.isdir()`

`os.path.exists()` tells you if something is there, but it does not specify whether that item is a regular file (like a CSV or Python file) or a folder (directory). To distinguish between the two, use:

- **`os.path.isfile(path)`:** Returns `True` only if the target is an actual regular file.
- **`os.path.isdir(path)`:** Returns `True` only if the target is a directory/folder.

```
import os

target_a = "data"
```

```
target_b = "data/raw/train_cats.csv"

print("Is 'data' a file?", os.path.isfile(target_a))
print("Is 'data' a directory?", os.path.isdir(target_a))

print("Is 'train_cats.csv' a file?", os.path.isfile(target_b))
print("Is 'train_cats.csv' a directory?", os.path.isdir(target_b))
```

**Expected Output:**

```
Is 'data' a file? False
Is 'data' a directory? True
Is 'train_cats.csv' a file? True
Is 'train_cats.csv' a directory? False
```

## 8. Joining Paths Safely: os.path.join()

One of the most frequent beginner mistakes is concatenating paths using string addition:

```
# ✗AVOID THIS ERROR-PRONE CODE:
folder = "datasets"
file = "cats.csv"
bad_path = folder + "/" + file # May fail on Windows or create invalid double slashes!
```

Why is string concatenation risky? Operating systems use different path separator characters:

- **Windows:** Uses the backslash character (\), e.g., datasets\cats.csv
- **Linux / macOS:** Uses the forward slash character (/), e.g., datasets/cats.csv

os.path.join() automatically detects the underlying operating system and inserts the correct separator, ensuring cross-platform compatibility:

```
import os

folder = "datasets"
subfolder = "raw"
filename = "cats.csv"

safe_path = os.path.join(folder, subfolder, filename)
print("Safely joined path:", safe_path)
```

**Expected Output:**

```
# On Linux / macOS:
Safely joined path: datasets/raw/cats.csv

# On Windows:
Safely joined path: datasets\raw\cats.csv
```

## 9. Creating Directories: `os.mkdir()` vs `os.makedirs()`

Python offers two functions for folder creation, and understanding the difference is vital:

- **`os.mkdir(path)`:** Creates a single directory. If parent folders along the path do not exist, it crashes with `FileNotFoundError`. If the target folder already exists, it crashes with `FileExistsError`.
- **`os.makedirs(path, exist_ok=True)`:** Recursively creates all intermediate parent directories needed. With `exist_ok=True`, it will not raise an error if the directory already exists.

```
import os

# Creating a single directory
if not os.path.exists("logs"):
    os.mkdir("logs")
    print("Created 'logs' folder.")

# Creating deep nested directories safely (Production Best Practice)
# If 'models' or 'checkpoints' or 'run_01' are missing, Python creates them all!
os.makedirs("models/checkpoints/run_01", exist_ok=True)
print("Nested checkpoint directory is ready.")
```

### Expected Output:

```
Created 'logs' folder.
Nested checkpoint directory is ready.
```

## 10. Removing Files: `os.remove()`

To permanently delete a file from the disk, use `os.remove()`.

```
import os

temp_file = "temporary_log.txt"

# Create a temporary file first for demonstration
with open(temp_file, "w") as f:
    f.write("Temporary debugging logs.")

# Delete the file
if os.path.exists(temp_file):
    os.remove(temp_file)
    print(f"Successfully deleted {temp_file}")
```

### Expected Output:

```
Successfully deleted temporary_log.txt
```

### ⚠ WARNING: Irreversible Deletion Warning

`os.remove()` permanently deletes files directly from the storage drive. It does NOT send items to the Windows

Recycle Bin or macOS Trash! Double-check all target paths before executing deletion commands.

## 11. Renaming and Moving Files: `os.rename()`

The `os.rename(source, destination)` function serves two purposes depending on whether the directory path changes:

- **Renaming:** When the parent folder remains identical but the filename changes.
- **Moving:** When the target location points to a different directory folder.

```
import os

# Example A: Renaming a file in the same directory
os.rename("draft_data.csv", "final_data.csv")
print("Renamed draft_data.csv to final_data.csv")

# Example B: Moving a file to another folder
# Assuming 'archive' directory exists:
os.rename("final_data.csv", os.path.join("archive", "final_data.csv"))
print("Moved file into archive/ folder")
```

## 12. Environment Variables: `os.getenv()` and `os.environ`

Environment variables are dynamic, system-wide key-value pairs maintained by the operating system outside your Python code. They are universally used in software development, cloud infrastructure, and AI engineering for two primary reasons:

- **1. Security:** Never hardcode passwords, database credentials, or secret API keys directly inside code repositories (such as GitHub). If a private repo is made public or shared, your credentials become compromised.
- **2. Configuration:** Different environments (development, testing, production) can provide distinct configurations without modifying a single line of Python source code.

Python provides two primary ways to access environment variables:

- **`os.environ`:** A dictionary-like object representing all system environment variables. Accessing a missing key like `os.environ['MISSING_KEY']` raises a `KeyError`.
- **`os.getenv(key, default=None)`:** A safe retrieval method. If the variable is not found, it returns `None` (or your custom default value) without crashing.

```
import os

# Reading an AI service API Key safely
api_key = os.getenv("OPENAI_API_KEY")

if api_key:
    print(f"API Key successfully retrieved: {api_key[:5]}...[SECURE]")
else:
    print("WARNING: OPENAI_API_KEY is not set in system environment!")
```

```
# Providing a sensible default fallback for non-secret configuration
database_host = os.getenv("DB_HOST", "localhost")
database_port = os.getenv("DB_PORT", "5432")

print(f"Connecting to database at {database_host}:{database_port}")
```

**Expected Output:**

```
WARNING: OPENAI_API_KEY is not set in system environment!
Connecting to database at localhost:5432
```

**⚠ WARNING: Security Rule for AI Students**

Never write: `api_key = 'sk-proj-123456789...'` in your Python files. Always load credentials dynamically from your system environment using `os.getenv()`.

## PART B — pathlib MODULE

While the `os` module is functional and historically prevalent, handling paths as plain strings frequently leads to clumsy syntax, cross-platform slashes bugs, and cumbersome string slicing. In Python 3.4, the `pathlib` module was introduced to provide an intuitive, modern, object-oriented approach.

### 13. Introduction to pathlib & Path Objects

In `pathlib`, filesystem paths are not strings—they are first-class Python objects. This means the path itself knows how to inspect its own properties, check its existence, create folders, and even read its file contents directly.

```
from pathlib import Path

# Create a Path object representing a file
p = Path("datasets/cats.csv")

print("Object representation:", repr(p))
print("Object type:", type(p))
```

**Expected Output:**

```
Object representation: PosixPath('datasets/cats.csv') # WindowsPath on Windows
Object type: <class 'pathlib.PosixPath'>
```

### 14. Current Working Directory: Path.cwd()

To get the current working directory as an object, call the classmethod `Path.cwd()`:

```
from pathlib import Path
```

```
current_dir = Path.cwd()
print("Current Working Directory:", current_dir)
print("Is it an instance of Path?", isinstance(current_dir, Path))
```

**Expected Output:**

```
Current Working Directory: /home/user/AI_Project
Is it an instance of Path? True
```

## 15. Creating Path Objects

You instantiate a Path object by passing strings or other paths into the Path() constructor:

```
from pathlib import Path

# Creating paths using relative strings
data_path = Path("data/raw/dataset.csv")

# Creating paths pointing to current directory
home_marker = Path(".")

print("Data path object:", data_path)
print("Type confirmation:", type(data_path))
```

## 16. Joining Paths Elegantly with the / Operator

pathlib overloads Python's division operator (/) to join path components cleanly and naturally. It is completely cross-platform, automatically formatting slashes correctly for Windows, macOS, and Linux.

```
from pathlib import Path

# Comparing traditional vs modern path joining:
# Traditional string way: os.path.join("AI", "datasets", "cats", "train")

# Modern pathlib way:
project_dir = Path("AI")
dataset_dir = project_dir / "datasets" / "cats" / "train"
file_target = dataset_dir / "batch_01.csv"

print("Composed Path:", file_target)
```

**Expected Output:**

```
# Linux / macOS:
Composed Path: AI/datasets/cats/train/batch_01.csv

# Windows:
Composed Path: AI\datasets\cats\train\batch_01.csv
```

**PRO TIP: Readability Advantage**

Using the / slash operator makes your code significantly more readable than nesting multiple arguments inside `os.path.join()`. At least one operand must be a Path object.

## 17. Checking Paths: exists(), is\_file(), and is\_dir()

Because a Path is an object, you query it directly using its methods rather than passing it into an external utility function:

```
from pathlib import Path

target = Path("data/raw/train_cats.csv")

print("Does target exist?      ", target.exists())
print("Is target a regular file? ", target.is_file())
print("Is target a directory?    ", target.is_dir())
```

**Expected Output:**

```
Does target exist?      True
Is target a regular file? True
Is target a directory?  False
```

## 18. Path Components: name, stem, suffix, and parent

Extracting filenames, extensions, and directory paths previously required complex string splitting. `pathlib` provides clean properties that break down any path effortlessly:

- **path.name:** The complete filename, including the extension (e.g., cats.csv).
- **path.stem:** The filename without the extension (e.g., cats).
- **path.suffix:** The file extension including the leading dot (e.g., .csv).
- **path.parent:** The logical directory containing the file (e.g., datasets).

Visual breakdown of datasets/cats.csv:

```

datasets / cats . csv
├── suffix → .csv
├── stem   → cats
├── name   → cats.csv
└── parent → datasets

```

```
from pathlib import Path

path = Path("datasets/cats.csv")

print("Full name: ", path.name)
```

```
print("File stem: ", path.stem)
print("Extension: ", path.suffix)
print("Parent dir: ", path.parent)
```

**Expected Output:**

```
Full name:  cats.csv
File stem:  cats
Extension:  .csv
Parent dir: datasets
```

## 19. Creating Directories: mkdir(parents=True, exist\_ok=True)

The mkdir() method on a Path object replaces both os.mkdir() and os.makedirs():

- **parents=True:** Automatically creates any missing parent folders in the chain (just like os.makedirs).
- **exist\_ok=True:** Prevents errors if the target directory already exists.

```
from pathlib import Path

# Safe directory creation
checkpoint_dir = Path("project/models/checkpoints")

# Creates 'project', 'models', and 'checkpoints' in one step without crashing
checkpoint_dir.mkdir(parents=True, exist_ok=True)
print(f"Directory {checkpoint_dir} verified or created.")
```

**Expected Output:**

```
Directory project/models/checkpoints verified or created.
```

## 20. Listing Directory Contents: iterdir()

To iterate over the files and subfolders within a directory, use .iterdir(). Unlike os.listdir() which only yields strings, .iterdir() yields full Path objects that are immediately ready for further inspection:

```
from pathlib import Path

data_dir = Path("data")

if data_dir.exists():
    for item in data_dir.iterdir():
        item_type = "Folder" if item.is_dir() else "File"
        print(f"[{item_type}] {item.name}")
```

**Expected Output:**

```
[Folder] raw
[Folder] processed
```

```
[File] metadata.json
```

## 21. Finding Files with Pattern Matching: glob()

In Data Science and Machine Learning, you frequently need to collect all files of a specific type (e.g., all CSV spreadsheets or all JPG images). The `glob()` method matches filenames using wildcard syntax:

- **\*.csv**: Matches any file ending in `.csv` in the target directory.
- **\*.jpg / \*.png**: Matches image formats.
- **\*.json**: Matches JSON configuration files.

```
from pathlib import Path

raw_dir = Path("data/raw")

print("Discovered CSV Datasets:")
for csv_file in raw_dir.glob("*.csv"):
    print(f" -> Found: {csv_file.name} (Full path: {csv_file})")
```

### Expected Output:

```
Discovered CSV Datasets:
-> Found: train_cats.csv (Full path: data/raw/train_cats.csv)
-> Found: train_dogs.csv (Full path: data/raw/train_dogs.csv)
```

## 22. Recursive File Searching: rglob()

While `glob()` searches only the immediate directory, `rglob()` ('recursive glob') searches through the current folder and all nested subfolders, no matter how deep they go. This is equivalent to `glob('**/*.ext')`.

```
from pathlib import Path

project_root = Path("AI_Project")

print("Searching recursively for all image files (.jpg):")
# Searches AI_Project/, data/raw/images/, models/test_images/, etc.
for img_path in project_root.rglob("*.jpg"):
    print("Found image at:", img_path)
```

### Expected Output:

```
Searching recursively for all image files (.jpg):
Found image at: AI_Project/data/raw/images/cat_001.jpg
Found image at: AI_Project/data/raw/images/cat_002.jpg
Found image at: AI_Project/data/raw/images/subfolder/dog_999.jpg
```

## 23. Reading and Writing Text Files Directly

pathlib provides convenient high-level convenience methods for quick text reading and writing without manually dealing with `open()`, `close()`, or context managers:

- **`path.write_text(content, encoding='utf-8')`**: Opens the file, writes the string content, and safely closes it.
- **`path.read_text(encoding='utf-8')`**: Opens the file, reads the entire content as a string, and closes it.

```
from pathlib import Path

notes_file = Path("experiment_notes.txt")

# Writing text in a single elegant line
notes_file.write_text("Model: ResNet50\nAccuracy: 94.6%\nStatus: Approved", encoding="utf-8")
print("Saved experiment notes.")

# Reading text in a single line
content = notes_file.read_text(encoding="utf-8")
print("\n--- File Contents ---")
print(content)
```

### Expected Output:

Saved experiment notes.

--- File Contents ---

Model: ResNet50  
Accuracy: 94.6%  
Status: Approved

## 24. Deleting Files: `unlink()`

To delete a file using pathlib, call the `.unlink()` method on the target Path object:

```
from pathlib import Path

temp_cache = Path("cache.tmp")

# Create a temporary file
temp_cache.write_text("Temporary cached computations.")

# Delete using unlink()
if temp_cache.exists():
    temp_cache.unlink()
    print("Cache file removed via unlink().")
```

### Expected Output:

Cache file removed via unlink().

## 25. Absolute vs Relative Paths

Understanding the difference between absolute and relative paths is fundamental in computing:

- **Relative Path:** Points to a file relative to the current working directory. If you change your working directory, the relative path might no longer point to the intended file. Example: `Path('data/cats.csv')`
- **Absolute Path:** Points to the exact, unambiguous location starting from the root of the filesystem. It works identically regardless of your current working directory. Example: `Path('/home/user/AI_Project/data/cats.csv')` on Linux or `Path('C:/Users/Student/AI_Project/data/cats.csv')` on Windows.

## 26. Resolving Full Paths: `resolve()`

The `.resolve()` method converts any relative path into an absolute, fully resolved canonical path on your system, expanding any symlinks or `'..'` symbols:

```
from pathlib import Path

rel_path = Path("data/raw/train_cats.csv")
abs_path = rel_path.resolve()

print("Relative path:", rel_path)
print("Resolved absolute path:", abs_path)
```

### Expected Output:

```
Relative path: data/raw/train_cats.csv
Resolved absolute path: /home/user/AI_Project/data/raw/train_cats.csv
```

## PART C — os VS pathlib COMPARISON

Use the comprehensive comparison table below as a quick guide when deciding which syntax to implement:

| Task / Operation          | Traditional os Syntax (Strings)               | Modern pathlib Syntax (Objects) |
|---------------------------|-----------------------------------------------|---------------------------------|
| Current Working Directory | <code>os.getcwd()</code>                      | <code>Path.cwd()</code>         |
| List Directory Contents   | <code>os.listdir(path)</code>                 | <code>path.iterdir()</code>     |
| Join Path Components      | <code>os.path.join(folder, "file.csv")</code> | <code>path / "file.csv"</code>  |
| Check if Path Exists      | <code>os.path.exists(path)</code>             | <code>path.exists()</code>      |
| Check if Regular File     | <code>os.path.isfile(path)</code>             | <code>path.is_file()</code>     |
| Check if Directory        | <code>os.path.isdir(path)</code>              | <code>path.is_dir()</code>      |

|                           |                                                         |                                                      |
|---------------------------|---------------------------------------------------------|------------------------------------------------------|
| Create Single Directory   | <code>os.mkdir(path)</code>                             | <code>path.mkdir()</code>                            |
| Create Nested Directories | <code>os.makedirs(path, exist_ok=True)</code>           | <code>path.mkdir(parents=True, exist_ok=True)</code> |
| Remove / Delete File      | <code>os.remove(path)</code>                            | <code>path.unlink()</code>                           |
| Rename / Move Path        | <code>os.rename(src, dst)</code>                        | <code>src.rename(dst)</code>                         |
| Find Files by Pattern     | <code>glob.glob('*.*csv')</code>                        | <code>path.glob('*.*csv')</code>                     |
| Find Files Recursively    | <code>os.walk()</code> / <code>glob('**/*.*csv')</code> | <code>path.rglob('*.*csv')</code>                    |
| Read Entire Text File     | <code>with open(path) as f: f.read()</code>             | <code>path.read_text()</code>                        |
| Write Text to File        | <code>with open(path, 'w') as f: f.write(s)</code>      | <code>path.write_text(s)</code>                      |
| Get Absolute Path         | <code>os.path.abspath(path)</code>                      | <code>path.resolve()</code>                          |
| Get Filename with Ext     | <code>os.path.basename(path)</code>                     | <code>path.name</code>                               |
| Get Parent Directory      | <code>os.path.dirname(path)</code>                      | <code>path.parent</code>                             |
| Get Environment Variable  | <code>os.getenv('KEY')</code>                           | Not in pathlib (Use <code>os.getenv</code> )         |
| Change Process Directory  | <code>os.chdir(path)</code>                             | Not in pathlib (Use <code>os.chdir</code> )          |

## PART D — PRACTICAL AI / DATA SCIENCE WORKFLOW

In professional Data Science and Machine Learning pipelines, maintaining clean and robust project paths ensures reproducibility across different operating systems and cloud environments.

### Automated Pipeline Directory Setup

Here is a realistic pattern used in Machine Learning repositories to establish paths, verify structure, inspect incoming datasets, and store processed artifacts:

```
from pathlib import Path

# 1. Define robust base project directories
BASE_DIR = Path("AI_Project")
RAW_DATA = BASE_DIR / "data" / "raw"
PROCESSED_DATA = BASE_DIR / "data" / "processed"
MODELS_DIR = BASE_DIR / "models" / "checkpoints"
```

```
# 2. Ensure essential output directories exist before training runs
PROCESSED_DATA.mkdir(parents=True, exist_ok=True)
MODELS_DIR.mkdir(parents=True, exist_ok=True)

print("Base project paths configured:")
print(" - Raw data folder:      ", RAW_DATA)
print(" - Processed output:      ", PROCESSED_DATA)
print(" - Model checkpoints:      ", MODELS_DIR)

# 3. Discover and inspect raw CSV datasets
print("\nScanning for raw datasets to process:")
csv_files = list(RAW_DATA.glob("*.csv"))

if not csv_files:
    print("No CSV files found in raw directory!")
else:
    for file in csv_files:
        print(f" -> Found dataset: {file.name} (Base name: {file.stem}, Suffix: {file.suffix})")
        # In a real pipeline, you would read pandas dataframe:
        # df = pd.read_csv(file)
        # df_clean = clean_pipeline(df)
        # df_clean.to_parquet(PROCESSED_DATA / f"{file.stem}.parquet")
```

**Expected Output:**

Base project paths configured:

- Raw data folder:        AI\_Project/data/raw
- Processed output:      AI\_Project/data/processed
- Model checkpoints:    AI\_Project/models/checkpoints

Scanning for raw datasets to process:

- > Found dataset: train\_cats.csv (Base name: train\_cats, Suffix: .csv)
- > Found dataset: train\_dogs.csv (Base name: train\_dogs, Suffix: .csv)

## PART E — QUICK REFERENCE CHEAT SHEETS

### os Module Cheat Sheet

| Function / Attribute              | One-Line Purpose & Explanation                                                |
|-----------------------------------|-------------------------------------------------------------------------------|
| <code>os.getcwd()</code>          | Returns the current working directory string of the running Python process.   |
| <code>os.chdir(path)</code>       | Changes the current working directory of the process to the specified path.   |
| <code>os.listdir(path='.')</code> | Returns a list of file and directory names contained in the target directory. |

|                                          |                                                                                                     |
|------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>os.path.exists(path)</code>        | Returns True if the specified file or directory path physically exists.                             |
| <code>os.path.isfile(path)</code>        | Returns True if the specified path points to an existing regular file.                              |
| <code>os.path.isdir(path)</code>         | Returns True if the specified path points to an existing directory.                                 |
| <code>os.path.join(p1, p2, ...)</code>   | Joins one or more path components using the OS-specific separator (/ or \).                         |
| <code>os.getenv(key, default)</code>     | Safely gets an environment variable value, or returns default if not found.                         |
| <code>os.environ</code>                  | Dictionary representing all system environment variables (raises <code>KeyError</code> if missing). |
| <code>os.mkdir(path)</code>              | Creates a single new directory; fails if parent folders are missing or folder exists.               |
| <code>os.makedirs(path, exist_ok)</code> | Creates nested directory hierarchies; <code>exist_ok=True</code> suppresses exists errors.          |
| <code>os.remove(path)</code>             | Permanently deletes the specified file from the disk (irreversible).                                |
| <code>os.rename(src, dst)</code>         | Renames or moves a file/folder from <code>src</code> to <code>dst</code> destination.               |

## pathlib Module Cheat Sheet

| Method / Property             | One-Line Purpose & Explanation                                               |
|-------------------------------|------------------------------------------------------------------------------|
| <code>Path.cwd()</code>       | Returns a Path object representing the current working directory.            |
| <code>Path('path/str')</code> | Constructs a new Path object from a path string or multiple path parts.      |
| <code>path1 / path2</code>    | Joins paths cleanly using the division operator with OS-correct slashes.     |
| <code>path.exists()</code>    | Returns True if the path object points to an existing file or directory.     |
| <code>path.is_file()</code>   | Returns True if the path exists and is a regular file.                       |
| <code>path.is_dir()</code>    | Returns True if the path exists and is a directory.                          |
| <code>path.name</code>        | Property: Returns the final path component filename including the extension. |
| <code>path.stem</code>        | Property: Returns the filename excluding the extension suffix.               |

|                                              |                                                                                                        |
|----------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <code>path.suffix</code>                     | Property: Returns the file extension including the leading dot (e.g., '.csv').                         |
| <code>path.parent</code>                     | Property: Returns the parent directory containing this path.                                           |
| <code>path.mkdir(parents, exist_ok)</code>   | Creates directory; <code>parents=True</code> builds parents, <code>exist_ok=True</code> avoids errors. |
| <code>path.iterdir()</code>                  | Yields Path objects for each file and directory inside the target folder.                              |
| <code>path.glob(pattern)</code>              | Yields Path objects matching wildcard pattern in the current folder.                                   |
| <code>path.rglob(pattern)</code>             | Recursively searches target folder and all subfolders for matching pattern.                            |
| <code>path.read_text(encoding)</code>        | Opens file, reads the entire content as string, and closes automatically.                              |
| <code>path.write_text(data, encoding)</code> | Opens file, writes string data to disk, and closes automatically.                                      |
| <code>path.unlink()</code>                   | Permanently deletes the file associated with this Path object.                                         |
| <code>path.resolve()</code>                  | Resolves any relative path and symlinks to return the canonical absolute Path.                         |

## PART F — PRACTICE LAB & EXERCISES

Solidify your mastery by completing the exercises below in your Python environment. Attempt these independently before reviewing reference notes!

### Beginner Student Exercises

- **Exercise 1 (CWD Inspection):** Write a script that prints the current working directory first using `os`, and then using `pathlib`. Confirm that both point to the same location.
- **Exercise 2 (Directory Audit):** Write a program that lists every file and folder in the current directory and prints whether each item is a file or a folder.
- **Exercise 3 (Safe Existence Check):** Define a variable `filename = 'dataset.csv'`. Check if this file exists on disk. If it exists, print its absolute resolved path; if not, print a friendly warning.
- **Exercise 4 (Nested Directory Builder):** Using `pathlib`, create a nested directory structure `'experiments/run_01/logs'` in a single command without crashing if it already exists.
- **Exercise 5 (File Creation & Inspection):** Inside `'experiments/run_01/logs'`, create a file called `'metrics.txt'` containing `'accuracy: 0.95'`. Use `pathlib` to read and print the contents.
- **Exercise 6 (Path Dissection):** Given the path `'data/financial/2026/annual_report.final.csv'`, print its parent folder, complete name, stem, and suffix using `pathlib` properties.
- **Exercise 7 (Pattern Matching):** Write a script using `.glob()` that prints only Python (`.py`) files located in your current folder.

- **Exercise 8 (Deep Recursive Discovery):** Using `.rglob()`, write a function that finds all image files (`.jpg`, `.png`) located anywhere inside a project folder and prints their file stems.
- **Exercise 9 (Environment Configuration):** Retrieve an environment variable named `'MODEL_ENV'`. If it is not set, default its value to `'development'` and print the active configuration.
- **Exercise 10 (Safe File Cleanup):** Create a temporary file named `'cleanup_test.tmp'` with some sample text. Verify it exists, delete it using `.unlink()`, and verify that it no longer exists.

## Mini Project: Dataset File Explorer

Practical Assignment: Build a modular command-line Python script named `dataset_explorer.py` that organizes and analyzes an input directory of datasets.

### Assignment Specifications & Requirements:

- **1. Input Parameter:** The program should accept a directory path as input (e.g., `datasets/` or `AI_Project/`).
- **2. Existence Verification:** Check whether the specified path exists and verify that it is actually a directory. If not, exit gracefully with an informative error message.
- **3. Comprehensive Discovery:** Iterate through the directory and count the total number of items, distinguishing regular files from directories.
- **4. CSV Metric Aggregation:** Find all CSV files (`.csv`) in the directory and print their names and byte sizes.
- **5. Recursive Media Scan:** Search recursively across all nested subdirectories for image files (`.jpg`, `.png`) and display each image's stem, extension, and parent folder.
- **6. Categorization / Separation:** Group all discovered files by their file extension (`.csv`, `.json`, `.txt`, `.png`, etc.) and print a summary table showing file counts per extension.
- **7. Pipeline Output Preparation:** Check whether a subfolder named `'processed/'` exists inside the target directory. If it does not exist, create it automatically so it is prepared for downstream training pipelines.

#### PRO TIP: Assignment Guidance

Strive to use `pathlib` for all path manipulation, searches, and directory creation in this project. Use `os` only where necessary (such as fetching environment configurations or reading system environment). Ensure your code is clean, commented, and functions across both Windows and macOS/Linux systems.